Universal Bridge (URIB)
Canonical Tokenization Specification

Structure-Layer Protocol Specification for Deterministic Token Canonicalization

Status	
Date	July 2026
Discipline	URIB Tokenization
Category	Technical Specification
Audience	Senior Software Engineers, Protocol Architects
Document ID	URIB-SPEC-CANON-1.0.0
Supersedes	None (Initial Release)

Abstract

This document specifies the Universal Bridge (URIB) Canonical Tokenization Specification, Version 1.0.0. It defines the Canonical Document Model (CDM), deterministic serialization rules, canonical hashing procedures using SHA3-256, lineage and versioning disciplines, reversible mapping guarantees, and schema-binding requirements for URIB token implementations. This specification applies to all domains including financial instruments, healthcare records, identity assets, supply chain artifacts, and IoT telemetry. Conformance levels are defined in Section 10. All requirements in this document are normative unless explicitly marked informative.

Table of Contents

1 Introduction and Scope

1.1 Purpose

The Universal Bridge (URIB) Canonical Tokenization Specification defines the rules, algorithms, and structural constraints required to produce a *canonical token* — a deterministic, byte-stable, self-describing digital representation of any logical asset instance across any information domain. This specification exists to eliminate the ambiguity that arises when the same logical asset is represented across multiple systems, schemas, serialization formats, or time periods.

In distributed systems, the same asset — an equity position, a patient encounter record, a supply chain shipment manifest, or an IoT device state snapshot — may be represented in dozens of source-system formats, each with its own field ordering conventions, encoding choices, numeric precision rules, and datetime representations. Without a canonical layer, two systems encoding semantically identical data will produce different byte sequences, resulting in different hash values. This non-determinism has cascading consequences: integrity verification fails across system boundaries; lineage chains break when parent digests cannot be reliably reproduced; schema-binding validation is inconsistent when field presence and ordering are not governed by a single normative contract; and audit trails are rendered unreliable for regulatory or forensic purposes.

Canonical tokenization resolves this by establishing a single, unambiguous serialization discipline. The canonical form of a token is defined as the one and only byte sequence that a compliant implementation will produce for a given logical asset state. All other representations — however semantically equivalent — are non-canonical and MUST NOT be used as the basis for digest computation, lineage anchoring, or schema validation within the URIB platform.

The canonical form serves three foundational roles: (1) it is the **source of truth** for hash computation and integrity verification; (2) it is the **anchor point** for lineage chain construction, ensuring that every mutation of an asset can be cryptographically traced back to its genesis; and (3) it is the **binding contract** between a token and its governing schema, ensuring that field-level semantics are unambiguous across all implementing systems and versioned schema boundaries.

Informative — Design Motivation

The discipline encoded in this specification was developed in response to observed production failures where logically identical asset records produced divergent digests due to: (a) inconsistent key ordering between serializers; (b) whitespace differences introduced by pretty-printing; (c) floating-point representation drift between languages; and (d) timezone encoding ambiguities in datetime fields. Each of these failure modes is addressed by a specific, mandatory rule in Section 3.

1.2 Scope

This specification is normative for all implementations of the URIB tokenization platform at Version 1.0.0. It governs the following technical domains:

1. **The URIB Canonical Document Model (CDM)** — the top-level structural schema that every URIB token MUST conform to, including the mandatory `meta`, `payload`, `lineage`, and `digest` blocks.
2. **Deterministic serialization rules** — the precise, step-ordered algorithm for transforming a token object into a byte sequence that is identical across all compliant implementations, regardless of programming language, operating system, or runtime environment.
3. **Canonical hashing algorithm and digest computation** — the use of SHA3-256 (FIPS 202) as the sole normative hash function for URIB canonical digests, including the exact procedure for digest computation and verification.
4. **Lineage and versioning rules** — the rules governing token ancestry, version increment discipline, delta chaining, branch/merge semantics, and the finality of the RETIRE operation.
5. **Reversible mapping guarantees** — the requirements for bijective transformation between canonical form and domain-native source-system representations, including type coercion rules and precision constraints.
6. **Schema-binding requirements** — the format and content requirements for URIB-compatible JSON Schemas, including the use of URIB extension keywords (`x-urib-*`), schema registry requirements, and the mandatory payload validation sequence.

7. **Implementation conformance levels** — the three-tier conformance model (Basic, Full, Extended) and the associated test vectors and rejection requirements.

This specification does NOT govern: transport protocols, authentication mechanisms, network topology, storage formats, or domain-specific payload semantics beyond those expressed through the schema-binding mechanism defined in Section 7. Those concerns are addressed by companion specifications in the URIB platform family.

Normative — Applicability

All implementing systems that produce, consume, validate, or relay URIB tokens MUST conform to this specification at or above Level 1 conformance as defined in Section 10.1. Systems claiming URIB compatibility MUST NOT implement a subset of these rules selectively; partial conformance constitutes non-conformance.

1.3 Normative References

The following documents are normative references for this specification. Implementers MUST treat these as authoritative sources for the behaviors described herein. Where this specification and a normative reference appear to conflict, this specification defines the URIB-specific constraint and the normative reference defines the underlying mechanism; implementers MUST satisfy both.

Reference	Document	Relevance
RFC 8259	The JavaScript Object Notation (JSON) Data Interchange Format	Base serialization format for all URIB tokens
FIPS 202	SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions (NIST)	Normative hash algorithm for canonical digest computation
RFC 3629	UTF-8, a transformation format of ISO 10646	Mandatory character encoding for all token serializations
ISO 8601	Data elements and interchange formats — Representation of dates and times	Normative datetime format for all temporal fields in canonical tokens

Reference	Document	Relevance
Semantic Versioning 2.0.0	Semantic Versioning Specification (semver.org)	Version format for `urib_version` and all `schema_id` version components
JSON Schema draft-07	JSON Schema: A Vocabulary for Structural Validation of JSON (draft-handrews-json-schema-01)	Base schema language for all URIB schema definitions
RFC 4648 §5	The Base16, Base32, and Base64 Data Encodings — Base 64 Encoding with URL and Filename Safe Alphabet	Base64url encoding for binary token transport (where applicable)
Unicode Standard	Unicode Standard Annex #15: Unicode Normalization Forms (NFC)	String normalization form mandated for all string primitives

1.4 Key Terms and Definitions

The following terms are defined for the purposes of this specification. Defined terms appear in **bold** at first use in each section. All RFC 2119 normative keywords (MUST, MUST NOT, SHALL, SHALL NOT, SHOULD, SHOULD NOT, MAY, RECOMMENDED, OPTIONAL) are used as defined in that document and appear in all-capitals throughout.

Term	Definition
Token	A URIB-canonical unit of information representing exactly one logical asset instance at one point in its versioned lifecycle. A token is a JSON object conforming to the CDM structure defined in Section 2.
Canonical Form	The unique, deterministic, byte-stable serialized representation of a token, produced by applying the canonical serialization algorithm defined in Section 3 to a fully populated token object. For any given logical asset state, exactly one canonical form exists.
Primitive	An atomic, irreducible field value within a token payload — specifically: a JSON string, number, boolean, or null. Objects and arrays are composed of primitives but are not themselves primitives.
Digest	The canonical SHA3-256 hash of a token's canonical serialization, expressed as a 64-character lowercase hexadecimal string. The digest uniquely identifies a token version and is the basis for all lineage and integrity operations.
Lineage Chain	An ordered, singly-linked sequence of token digests tracing the complete mutation history of a logical asset from its genesis token to its most recent version. Each link in the chain is expressed as a `parent_digest` reference.
Anchor	The first (genesis) token in a lineage chain, identified by `lineage.version == 1` and `lineage.parent_digest == null`. The anchor's digest is the permanent Asset ID for the asset's lifetime.
Schema Binding	The association of a token's `payload` block to a specific, versioned JSON Schema identified by `meta.schema_id`. Schema binding governs field presence, types, constraints, and URIB-specific mapping metadata.

Term	Definition
Domain-Native Form	The source-system representation of an asset prior to canonicalization — for example, a raw database record, a proprietary message format, or a vendor-specific JSON document. Domain-native forms are not themselves URIB tokens.
Reversible Mapping	A bijective (one-to-one and onto) transformation pair — `(canonicalize, restore)` — between a canonical token's payload and its domain-native form, such that no semantic information is lost in either direction of transformation.
CDM	Canonical Document Model. The top-level structural schema defined in Section 2 that every URIB token MUST conform to, comprising the `meta`, `payload`, `lineage`, and `digest` top-level blocks.
Asset ID	The digest of the genesis token for a given logical asset. The Asset ID is immutable and persists as `meta.asset_id` across all versions of a token's lineage chain.
Delta Object	A separately stored, canonicalized JSON object describing the field-level diff between a token's payload and its parent's payload. Referenced by digest via `lineage.delta_ref`.
Genesis Token	Synonymous with Anchor. The version-1 token that establishes a new asset in the URIB system. Its digest becomes the permanent Asset ID.
Unicode Code Point Order	Lexicographic ordering of strings by the integer value of their Unicode code points, equivalent to UTF-16 code unit ordering for characters in the Basic Multilingual Plane. Used for deterministic key ordering in canonical serialization.

2 Canonical Document Model (CDM)

2.1 Overview

The Canonical Document Model (CDM) defines the top-level structure that every URIB token MUST conform to. The CDM is the structural contract that enables interoperability across domains, implementations, and versioned schema boundaries. Adherence to the CDM is a prerequisite for digest computation, lineage chain construction, schema binding, and all conformance-level behaviors defined in this specification.

A URIB token is a JSON object (as defined by RFC 8259) that MUST contain exactly four top-level keys: `meta`, `payload`, `lineage`, and `digest`. No additional top-level keys are permitted. The presence of any key not defined in this section at the top level of a token constitutes a CDM violation and the token MUST be rejected.

Normative — CDM Completeness

All four top-level keys — `meta`, `payload`, `lineage`, and `digest` — MUST be present in every token, even if `payload` is an empty object `{}` (as in a RETIRE token). Omission of any top-level key constitutes a CDM violation.

2.2 Top-Level Structure

The following annotated JSON template defines the normative CDM structure. Each field is described in detail in the subsections below. Comments within the template are for documentation purposes only; the canonical serialization of an actual token MUST NOT contain comments.

```
{   "meta": {     "urib_version": "1.0.0",           // Spec version
implementing this token     "token_id":     "<uuidv7>",        // Time-
ordered UUID, unique per token instance     "asset_id":
"<genesis_digest_hex>",  // SHA3-256 hex of genesis token     "schema_id":
"<namespace>:<schema_name>:<semver>",     "domain":
"<domain_identifier>",   // See Appendix A     "created_at":   "<ISO 8601 UTC
with milliseconds>",     "issued_by":    "<issuer_urn>"   },   "payload": {
// Domain-specific fields governed entirely by schema_id     // All fields
canonicalized per Section 3   },   "lineage": {     "version":       1,
// Integer, monotonically increasing from 1     "parent_digest": null,
// SHA3-256 hex of parent token; null at genesis     "branch":        null,
// Branch label; null on mainline     "operation":     "CREATE",          //
CREATE | UPDATE | MERGE | RETIRE     "delta_ref":     null               //
SHA3-256 hex of delta object; null if no delta   },   "digest": "<sha3-256-
hex>"           // 64-char lowercase hex; computed last }
```

The following table provides the authoritative field-level specification for all CDM fields across all four top-level blocks. Type, requiredness, constraints, and semantics are normative.

Block	Field	Type	Required	Constraints	Description
`meta`	**urib_version**	string	MUST	SemVer 2.0.0 format	The version of the URIB specification this token was issued under. MUST match the implementing spec version exactly.
`meta`	**token_id**	string	MUST	UUIDv7, lowercase, hyphenated	Globally unique identifier for this specific token instance. Time-ordered for indexing efficiency. Uniqueness MUST be verified on ingest.
`meta`	**asset_id**	string	MUST	64-char lowercase hex	The SHA3-256 digest of the genesis token for this asset.

Block	Field	Type	Required	Constraints	Description
					Immutable across all versions. At genesis, MUST equal the token's own digest.
meta	**schema_id**	string	MUST	Format: ns:name:semver	Colon-delimited identifier referencing the versioned JSON Schema governing this token's payload. See Section 7.2.
meta	**domain**	string	MUST	Dot-delimited identifier; see Appendix A	Enumerated domain identifier. MUST match the x-urib-domain value declared in the referenced schema.
meta	**created_at**	string	MUST	ISO 8601, millisecond precision, UTC (Z suffix)	The UTC timestamp at which this token version was issued. MUST include millisecond precision. MUST use Z suffix (not +00:00).
meta	**issued_by**	string	MUST	URN format	A URN identifying the system, service, or authority that issued this token. Format: urn:urib:issuer:<identifier>.
payload	*(domain fields)*	object	MUST	Governed by schema_id	Domain-specific content. Field set, types, and constraints are entirely governed by the referenced JSON Schema. May be {} for RETIRE tokens.
lineage	**version**	integer	MUST	Monotonically increasing, >= 1	The version number of this token within its lineage chain. MUST be 1 for genesis tokens. MUST increment by exactly 1 for each subsequent version.
lineage	**parent_digest**	string|null	MUST	64-char lowercase hex or null	The digest of the immediately preceding token version. MUST be null at genesis. MUST reference a verified, existing token for all subsequent versions.
lineage	**branch**	string|null	MUST	Non-empty string or null; unique within asset	Optional branch label for fork scenarios. MUST be null on the mainline. MUST be unique within the asset's lineage if non-null.
lineage	**operation**	string	MUST	Enum: CREATE, UPDATE, MERGE, RETIRE	The type of operation this token represents. See Section 5.4 for full semantics of each operation value.
lineage	**delta_ref**	string|null	MUST	64-char lowercase hex or null	The digest of a separately stored delta object describing the field-level diff from parent to current.

Block	Field	Type	Required	Constraints	Description
					See Section 5.6. Null if no delta was computed.
top-level	**digest**	string	MUST	64-char lowercase hex	The SHA3-256 hash of the canonical serialization of this token with the digest field substituted as null. Computed last. MUST be verified on receipt.

2.3 The meta Block

The `meta` block provides token-level metadata that enables routing, versioning, schema resolution, and provenance tracking. Every field in the `meta` block is mandatory. No additional fields are permitted within `meta`.

`urib_version` (string, SemVer): Identifies the version of this specification under which the token was produced. Implementations MUST reject tokens whose `urib_version` is not supported by the receiving system. A receiving system supporting version 1.0.0 MUST accept tokens with `urib_version` equal to `"1.0.0"` and MAY accept tokens from earlier minor/patch versions if backward compatibility is established. Cross-major-version acceptance MUST NOT be assumed.

`token_id` (string, UUIDv7): A time-ordered globally unique identifier for this specific token instance. UUIDv7 is mandated (rather than UUIDv4) to enable time-based indexing without secondary timestamp indexes. The value MUST be lowercase and hyphenated in canonical 8-4-4-4-12 form (e.g., `018f2e3a-5b1c-7d2e-9f4a-0b3c8d1e6f2a`). Implementations MUST maintain a deduplication store keyed on `token_id` and MUST reject tokens whose `token_id` has already been seen within the configured retention window.

`asset_id` (string, hex): The SHA3-256 digest of the genesis token (version 1) for the logical asset this token represents. This value is immutable for the lifetime of the asset — it MUST be identical across all token versions in a lineage chain. At genesis, `asset_id` MUST equal the token's own `digest` field. For all subsequent versions, the `asset_id` MUST match the genesis token's digest, enabling cross-version lookup without full lineage traversal.

`schema_id` (string): A colon-delimited three-part identifier in the form `<namespace>:<schema_name>:<semver>` that references the versioned JSON Schema governing this

token's payload. The namespace MUST be a dot-delimited domain hierarchy. The schema name MUST be in `snake_case`. The version MUST be SemVer 2.0.0. See Section 7.2 for full format rules and examples.

`domain` (string): An enumerated dot-delimited identifier designating the domain to which this token belongs. Reserved domain identifiers are defined in Appendix A. The `domain` value MUST match the `x-urib-domain` extension field declared in the schema referenced by `schema_id`. Discrepancies between `meta.domain` and the schema's declared domain MUST cause the token to be rejected.

`created_at` (string, ISO 8601): The UTC timestamp at which this token version was issued. The value MUST include millisecond precision (three decimal places in the seconds component) and MUST terminate with the `Z` suffix denoting UTC. The format is: `YYYY-MM-DDTHH:mm:ss.sssZ`. The `+00:00` offset notation MUST NOT be used as an alternative to `Z` in canonical form.

`issued_by` (string, URN): A URN identifying the issuing system or authority. The RECOMMENDED format is `urn:urib:issuer:<identifier>` where `<identifier>` is a system-specific, URL-safe, non-empty string. The issuer URN enables attribution and access-control auditing. Implementations MUST validate that the issuer is authorized to issue tokens for the declared domain.

2.4 The payload Block

The `payload` block contains the domain-specific content of the token. Its structure and field set are entirely governed by the JSON Schema referenced in `meta.schema_id`. The following rules apply to all payload content regardless of domain:

8. All string values within `payload` MUST be encoded as UTF-8 and MUST be Unicode NFC-normalized (Unicode Normal Form C) prior to serialization. Unnormalized string values MUST be rejected.
9. All numeric values MUST be represented as JSON numbers. The canonical numeric representation rules defined in Section 3.2 (Step 5) MUST be applied to all numeric payload fields.
10. Fields declared in the schema as required but carrying a null value MUST be explicitly present as JSON `null`. Omitting a required-but-null field is a CDM violation. Optional fields that are absent

MUST be omitted entirely; they MUST NOT be represented as `null` unless the schema explicitly permits null as a valid value for that field.

11. Nested objects within the payload follow the same canonicalization rules recursively. Key ordering, string normalization, and numeric representation apply at every level of nesting without exception.

12. Array fields MUST preserve the domain-semantic ordering of their elements. Arbitrary re-sorting of array elements is PROHIBITED unless the schema explicitly declares the array as order-insensitive using the `x-urib-order-insensitive: true` extension. For order-insensitive arrays, the canonical sort order is Unicode code point order of the JSON-serialized element.

> **Warning — Payload Completeness**
>
> Implementations MUST NOT add fields to the payload that are not declared in the referenced schema. The schema MUST declare `"additionalProperties": false`. Any token whose payload contains fields not present in the resolved schema MUST be rejected at validation time (see Section 7.6).

2.5 The lineage Block

The `lineage` block encodes the token's position within its asset's version history. It is the structural mechanism by which the immutable lineage chain is constructed and verified. All five fields within `lineage` MUST be present.

`version` (integer): The version number of this token within its lineage chain. MUST be a positive integer greater than or equal to 1. The genesis token MUST carry version 1. Each successive version MUST increment by exactly 1. Version gaps are explicitly prohibited and constitute a lineage integrity violation.

`parent_digest` (string or null): The SHA3-256 digest of the immediately preceding token version, expressed as a 64-character lowercase hex string. At genesis (version 1), this field MUST be the JSON literal `null`. For all versions greater than 1, this field MUST reference a token that: (a) has been verified by the receiving system, (b) carries version number equal to the current version minus one, and (c) carries the

same `meta.asset_id`. For MERGE tokens, see the extended parent_digests syntax defined in Section 5.4.

`branch` (string or null): An optional string label identifying the branch on which this token was issued. MUST be null on the mainline. If non-null, the label MUST be unique within the asset's lineage and MUST be a non-empty, URL-safe string. Branch versions are version-namespaced (e.g., `audit.1`, `review.2`) and their version integers are independent of the mainline sequence.

`operation` (string): An enumerated value indicating the type of mutation this token represents. MUST be one of: `CREATE`, `UPDATE`, `MERGE`, or `RETIRE`. Full semantics for each operation value are defined in Section 5.4.

`delta_ref` (string or null): The SHA3-256 digest of a separately stored and independently canonicalized delta object that describes the field-level diff between this token's payload and its parent's payload. If no delta object was produced, this field MUST be the JSON literal `null`. The delta object itself is defined in Section 5.6. Implementations at Level 3 conformance MUST populate this field for all UPDATE and MERGE operations.

2.6 The digest Field

The `digest` field is a top-level field (not nested within any block) that contains the SHA3-256 hash of the canonical serialization of the token. It is the central integrity mechanism of the URIB platform and MUST be treated with the highest priority in any processing pipeline.

The digest is computed LAST, after all other fields in the token have been finalized and no further mutations are intended. The computation procedure is defined in Section 4.2. The digest is expressed as a 64-character lowercase hexadecimal string (representing 32 bytes / 256 bits of hash output). No prefix (such as `0x`) is used.

Upon receipt of any token, the verifier MUST perform digest verification (Section 4.3) before proceeding with any other processing. A token whose digest does not match the computed digest of its canonical serialization MUST be rejected unconditionally. Silent acceptance of tokens with invalid digests is explicitly prohibited and constitutes a security violation.

Normative — Digest-First Processing

Digest verification (Section 4.3) MUST precede schema validation (Section 7.6) in all token processing pipelines. A token that passes schema validation but fails digest verification MUST be rejected. A token that fails digest verification MUST NOT be passed to schema validation or any downstream processor.

---

3 Deterministic Serialization Rules

3.1 Motivation

Deterministic serialization is the foundational prerequisite for canonical hashing. Two logically identical tokens MUST produce the same byte sequence when serialized, across all conforming implementations, in all programming languages, on all platforms. Without deterministic serialization, hash values diverge for identical logical content, breaking digest verification, lineage chain integrity, and all downstream operations that depend on digest equality.

The principal sources of non-determinism in JSON serialization are: (1) object key ordering, which is undefined by the JSON specification (RFC 8259); (2) whitespace — indentation, spaces around colons and commas, trailing newlines — which JSON parsers and formatters apply inconsistently; (3) floating-point number representation, which varies across languages and runtime implementations; (4) Unicode normalization form, which differs between source systems; and (5) the treatment of null versus absent for optional fields. Each of these sources is addressed by a specific normative step in the serialization algorithm below.

Informative — Why Not a Binary Format?

JSON was selected as the serialization format for URIB tokens because it is universally supported, human-readable, and amenable to schema validation with standard tooling. The determinism

requirements in this section constrain JSON serialization sufficiently to achieve byte-stable canonical forms without requiring a binary encoding layer.

3.2 Serialization Algorithm

The canonical serialization algorithm MUST be applied in the following step order. Each step is normative. Deviations from any step constitute a serialization violation and will produce a non-canonical byte sequence.

Step 1 — Character Encoding. The output of canonical serialization MUST be encoded as UTF-8 (RFC 3629) without a Byte Order Mark (BOM). A BOM (U+FEFF at the start of the byte stream) MUST NOT be present. Any input characters outside the valid UTF-8 range MUST be rejected before serialization begins.

Step 2 — Whitespace Elimination. No whitespace characters (U+0020 SPACE, U+0009 TAB, U+000A LINE FEED, U+000D CARRIAGE RETURN) MAY appear between any JSON tokens in the serialized output. This means: no spaces after colons, no spaces after commas, no indentation, no newlines between fields or at the end of the document. The canonical form is a single, unbroken line of UTF-8 bytes.

Step 3 — Key Ordering. All JSON object keys MUST be sorted in Unicode code point order (ascending lexicographic order by the integer values of their Unicode code points) at every level of object nesting. This ordering is applied recursively to all objects within the token, including objects nested within arrays, objects within the payload, and objects within the `meta`, `lineage`, and other blocks. The sort is stable for identical keys, but duplicate keys are PROHIBITED by RFC 8259 and MUST cause rejection.

Step 4 — String Normalization. All string values — in both keys and values — MUST be Unicode NFC-normalized before serialization. NFC (Canonical Decomposition, followed by Canonical Composition) is the Unicode Normal Form C as defined in Unicode Standard Annex #15. Implementations MUST apply NFC normalization to all strings before sorting keys and before serializing string values. Strings that are already in NFC form are unaffected.

Step 5 — Numeric Representation. JSON number values MUST be serialized according to the following canonical numeric rules:

- **Integers:** Serialized without a decimal point, without leading zeros (except for the value zero itself, which is `0`), and without a trailing decimal or exponent. The value 42 MUST be serialized as `42`, not `42.0` or `42.00`.
- **Decimal values:** Serialized using the shortest representation that preserves the full precision of the stored value. Trailing zeros after the decimal point MUST be omitted. The value 1.50 MUST be serialized as `1.5`. The value 195.42 MUST be serialized as `195.42`.
- **Scientific notation:** MUST NOT be used for values in the range `1e-6 <= |x| < 1e20`. MUST be used for values outside this range, in the form `<mantissa>e<exponent>` with no leading zeros in the exponent and a single-digit mantissa if possible.
- **Special values:** JSON does not support NaN or Infinity. Implementations MUST NOT serialize NaN or Infinity values; such values MUST be replaced with `null` and the schema MUST declare the field as nullable, or the token MUST be rejected before serialization.

Step 6 — Boolean and Null Literals. The JSON literals `true`, `false`, and `null` MUST be serialized in lowercase exactly as shown. No capitalization variants (`True`, `False`, `NULL`) are permitted.

Step 7 — Array Element Order. Array elements MUST be serialized in their original domain-semantic order, which is preserved from the source payload construction. Implementations MUST NOT sort array elements unless the schema explicitly declares the array as order-insensitive (see Section 2.4, rule 5). The canonical serialization of an order-insensitive array sorts elements by the Unicode code point ordering of the serialized JSON representation of each element.

Step 8 — Digest Placeholder. When performing canonical serialization for the purpose of digest computation, the `"digest"` field value MUST be replaced with the JSON literal `null` before serialization. The key `"digest"` MUST still be present in the serialized output (with value `null`), because removing the key would change the key ordering context of the top-level object. The `"digest"` key sorts before `"lineage"`, `"meta"`, and `"payload"` in Unicode code point order.

Step 9 — No Extensions. The canonical serialization MUST be strict, standard JSON as defined by RFC 8259. The following are explicitly prohibited: JavaScript-style comments (`//` and `/* */`); trailing commas after the last element of an object or array; single-quoted strings; unquoted keys; hexadecimal number literals; non-standard escape sequences; embedded NUL bytes.

Normative — Step Ordering

The nine steps of the canonical serialization algorithm MUST be applied in the order listed. In particular, Step 4 (NFC normalization) MUST be applied before Step 3 (key ordering), because NFC normalization may change the Unicode code points of a string, altering its sort position.

3.3 Serialization Reference Example

The following example illustrates the transformation from a non-canonical domain-native JSON representation to the canonical serialized form. This example is for illustration purposes and does not represent a complete token.

Non-canonical (domain-native) input:

```
{   "name"        : "Apple Inc.",   "price_usd"   : 195.42000,   "ticker"
: "AAPL",   "isin"        : "US0378331005",   "market_cap"  : null,   "tags"
: ["sp500", "technology", "large-cap"],   "active"      : true,   "exchange"
: "NASDAQ" }
```

Canonical serialized form (Step-by-Step transformation applied):

```
{"active":true,"exchange":"NASDAQ","isin":"US0378331005","market_cap":null,"n
ame":"Apple Inc.","price_usd":195.42,"tags":["sp500","technology","large-
cap"],"ticker":"AAPL"}
```

The following transformations were applied:

- **Key reordering (Step 3):** Keys are sorted in Unicode code point order: `active`, `exchange`, `isin`, `market_cap`, `name`, `price_usd`, `tags`, `ticker`.
- **Whitespace removal (Step 2):** All spaces, including those around the colon, have been removed.
- **Trailing zero removal (Step 5):** `195.42000` becomes `195.42`.
- **Null preserved (Step 6):** `market_cap: null` is retained as-is (field was present in source).

- **Array order preserved (Step 7):** The array element order is unchanged from the source.

> **Warning — Null vs. Absent**
>
> The `market_cap` field is present as `null` in both the source and canonical form. If the source had omitted this field entirely (i.e., the key was absent), and the schema declares it as required-nullable, the canonicalization step MUST NOT insert the missing key as null. Instead, the token MUST be rejected for schema violation before reaching the serialization step. The canonical serializer is not responsible for repairing schema-invalid tokens.

3.4 Prohibited Patterns

The following table enumerates prohibited serialization patterns. Any token exhibiting these patterns MUST be rejected by a conforming implementation.

Prohibited Pattern	Example	Reason
Keys not in Unicode code point order	`{"ticker":"AAPL","isin":"US..."}`	Violates Step 3; produces different byte sequence for identical content
Pretty-printed with whitespace	`{ "key" : "value" }`	Violates Step 2; whitespace between tokens produces divergent byte sequences
Float with trailing zeros	`195.42000`	Violates Step 5; trailing zeros alter byte sequence without semantic change
Integer with decimal point	`42.0` or `42.00`	Violates Step 5; integers MUST NOT carry a decimal point
Unnecessary scientific notation	`1.9542e2` for 195.42	Violates Step 5; scientific notation prohibited for values in normal range
Omitted null field	Key absent when schema declares it required-nullable	Violates Section 2.4 rule 3; null MUST be explicitly present
BOM prefix	UTF-8 byte stream starting with EF BB BF	Violates Step 1; BOM alters the byte sequence and is not valid in JSON

Prohibited Pattern	Example	Reason
Non-UTF-8 encoding	UTF-16, Latin-1, or Windows-1252 encoded output	Violates Step 1; only UTF-8 is the canonical encoding
JSON comments	`// comment` or `/* comment */`	Violates Step 9; non-standard JSON extensions are prohibited
Trailing commas	`{"a":1,"b":2,}`	Violates Step 9; trailing commas are not valid JSON per RFC 8259
Non-NFC string content	String with NFD-normalized characters	Violates Step 4; Unicode NFC normalization is mandatory for all strings
Digest field absent during hashing	Token serialized without `"digest"` key	Violates Step 8; `"digest":null` MUST be present during hash computation

4 Canonical Hashing Rules

4.1 Hash Algorithm

The canonical hash algorithm for all URIB digest computations is SHA3-256 as defined in NIST FIPS 202 (the Keccak-p[1600, 24] permutation with rate r=1088 and capacity c=512). SHA3-256 produces a 256-bit (32-byte) digest from an arbitrary-length byte sequence.

Implementations MUST use SHA3-256 exclusively for canonical digest computation. The following algorithms MUST NOT be used as substitutes for canonical URIB digests: SHA-256 (SHA-2 family), SHA-1, MD5, RIPEMD-160, BLAKE2, or any algorithm not explicitly designated as SHA3-256 per FIPS 202. The SHA-2 family of algorithms (including SHA-256) is explicitly prohibited despite its widespread use, because the URIB platform mandates SHA3 for its domain-separation properties and resistance to length-extension attacks inherent to the Merkle-Damgård construction used by SHA-2.

Normative — Algorithm Prohibition

Implementations MUST NOT accept tokens whose digest fields were computed using SHA-256, SHA-1, or any algorithm other than SHA3-256. An implementation that cannot verify whether the digest was computed with the correct algorithm MUST reject the token as unverifiable and log it as such.

Implementers SHOULD use a FIPS 202 certified library for SHA3-256 computation. In environments where FIPS certification is required, the SHA3-256 implementation MUST be validated under FIPS 140-2 or FIPS 140-3 as applicable.

4.2 Digest Computation Procedure

The digest of a URIB token is computed using the following normative procedure. All steps MUST be executed in the order listed. The pseudocode implementation of these steps is provided in Section 9.5.

13. **Populate all non-digest fields.** The token object MUST be fully populated with all `meta`, `payload`, and `lineage` fields before digest computation begins. No field other than `digest` may be modified after this step.
14. **Set the digest placeholder.** Set the value of the `"digest"` field to the JSON literal `null`. The key MUST remain present. This ensures the `"digest"` key participates in the canonical key ordering (it sorts before `"lineage"`, `"meta"`, and `"payload"`) and that the byte sequence length contributed by the digest field is consistent across all tokens.
15. **Apply canonical serialization.** Apply the canonical serialization algorithm defined in Section 3.2 (all nine steps, in order) to the working token object with `"digest": null`. The output is a UTF-8 byte sequence with no BOM.
16. **Compute SHA3-256.** Compute the SHA3-256 hash of the UTF-8 byte sequence produced in Step 3. The input to the hash function is the raw byte sequence, not a base64 or hex encoding of it. The output is a 32-byte binary digest.

17. **Encode the digest.** Encode the 32-byte binary digest as a lowercase hexadecimal string of exactly 64 characters. No separators, spaces, or prefix characters (such as `0x`) are used. Each byte is represented as exactly two hexadecimal digits, with leading zeros preserved.
18. **Write the digest into the token.** Assign the 64-character lowercase hex string computed in Step 5 to the `"digest"` field of the token. The token is now complete and the digest is authoritative.

4.3 Digest Verification

Upon receipt of a URIB token, a verifier MUST perform digest verification before any other processing. The verification procedure is as follows:

19. **Extract the claimed digest.** Read and store the value of the `"digest"` field from the received token. This is the claimed digest.
20. **Substitute the digest placeholder.** Set the `"digest"` field of the received token to the JSON literal `null`. Do not modify any other field.
21. **Re-serialize canonically.** Apply the canonical serialization algorithm (Section 3.2) to the token with the substituted null digest. The output is a UTF-8 byte sequence.
22. **Compute SHA3-256.** Compute SHA3-256 over the UTF-8 byte sequence from Step 3.
23. **Encode the computed digest.** Encode the 32-byte SHA3-256 output as a 64-character lowercase hex string.
24. **Compare digests.** Perform a byte-for-byte equality comparison between the computed digest (Step 5) and the claimed digest (Step 1). A constant-time comparison function SHOULD be used to prevent timing-based side-channel attacks.
25. **Accept or reject.** If the digests are equal, the token's integrity is confirmed and processing MAY proceed to schema validation (Section 7.6). If the digests are not equal, the token MUST be rejected immediately. The rejection MUST be logged with: the token's `token_id`, the `created_at` timestamp, the claimed digest, the computed digest, and the current UTC timestamp of the rejection event.

Warning — Rejection Logging

Logging of digest verification failures is mandatory, not optional. Silent rejection (discarding the token without a log record) defeats the ability to detect systematic tampering or misconfigured issuers. The log record MUST include all fields specified in Step 7 above.

4.4 Asset ID Derivation

The Asset ID is the permanent, immutable identifier for a logical asset across its entire lifecycle. It is derived from the genesis token as follows:

26. Issue the genesis token with `lineage.version = 1`, `lineage.parent_digest = null`, and `lineage.operation = "CREATE"`.
27. Compute the digest of the genesis token per Section 4.2.
28. The resulting 64-character hex digest is the Asset ID. It MUST be written into `meta.asset_id` of the genesis token itself (making the genesis token self-referential on the `asset_id` field).
29. All subsequent token versions in the lineage chain MUST carry this same Asset ID value in `meta.asset_id` without modification.

The Asset ID enables cross-version lookup: a query for an asset by its Asset ID retrieves all token versions without requiring traversal of the lineage chain. It also enables authoritative proof of asset identity: any party with the genesis token can independently compute and verify the Asset ID.

Normative — Asset ID Immutability

The Asset ID MUST NOT be changed once established by the genesis token. Any token claiming a different `asset_id` than the one computed from its chain's genesis token MUST be rejected as a lineage integrity violation.

4.5 Digest Stability Guarantee

Once a token's digest has been computed (Step 6 of Section 4.2), no field other than `digest` may be mutated without invalidating the digest. This constraint is the cornerstone of the URIB integrity model: a digest is not merely a checksum but a cryptographic commitment to the exact byte content of the token at the time of issuance.

If any field in a token must be changed after digest computation — for any reason including error correction, schema migration, or regulatory amendment — the implementer MUST NOT modify the existing token's fields. Instead, a new token version MUST be issued with the corrected values, a new digest computed, a new `token_id` assigned, an incremented `lineage.version`, and `lineage.parent_digest` set to the digest of the preceding version. The old token version remains in the lineage chain as an immutable historical record.

---

5 Lineage and Versioning Rules

5.1 Lineage Chain Structure

A lineage chain is a cryptographically linked, singly-linked list of URIB token digests that traces the complete mutation history of a logical asset from its genesis to its current state. Each token in the chain references the digest of its predecessor via the `lineage.parent_digest` field. The chain is anchored at the genesis token, whose `parent_digest` is null.

The structural integrity of a lineage chain is determined by the unbroken sequence of digest references. Any gap, cycle, or digest mismatch in this sequence constitutes a lineage integrity violation. The following diagram illustrates the canonical structure of a three-version mainline lineage chain:

[v1: digest=A | parent=null | op=CREATE] | | parent_digest=A v [v2: digest=B | parent=A | op=UPDATE] | | parent_digest=B v [v3: digest=C | parent=B | op=UPDATE]

In this chain, the Asset ID equals digest A (the genesis digest). Token v3's `meta.asset_id` equals digest A. To verify token v3's lineage, a verifier must recursively verify: that v3's `parent_digest` matches v2's digest, that v2's `parent_digest` matches v1's digest, and that v1 is a valid genesis token with a self-referential `asset_id`.

5.2 Genesis Token Rules

The genesis token is the first token in a lineage chain and is the authoritative origin of an asset's identity. A genesis token MUST satisfy all of the following conditions:

- `lineage.version` MUST be the integer `1`.
- `lineage.parent_digest` MUST be the JSON literal `null`.
- `lineage.operation` MUST be the string `"CREATE"`.
- `lineage.branch` MUST be the JSON literal `null` (genesis tokens are always on the mainline).
- `lineage.delta_ref` MUST be the JSON literal `null` (no parent to diff against).
- `meta.asset_id` MUST equal the token's own `digest` field. This self-referential relationship is the defining characteristic of a genesis token and MUST be verified by all compliant implementations upon receipt of a claimed genesis token.

Normative — Genesis Self-Reference Verification

When receiving a token claiming to be a genesis token (i.e., `lineage.version == 1` and `lineage.parent_digest == null`), implementations MUST verify that `meta.asset_id == digest` after performing digest verification (Section 4.3). A genesis token where `asset_id` does not equal its own computed digest MUST be rejected.

5.3 Version Increment Rules

The version numbering discipline within a lineage chain is strict and admits no exceptions. The following rules are normative for all non-genesis tokens:

30. Each token version MUST increment the `lineage.version` integer by exactly **one** relative to its parent token's `lineage.version`. A version increment of more than one (a gap) is explicitly prohibited.
31. Version numbers MUST be positive integers. Zero is not a valid version number. Negative integers are not valid version numbers.
32. The `lineage.parent_digest` field MUST reference a token that has been verified by the receiving system. Implementations MUST NOT accept tokens whose parent digest references an unknown or unverified token. An implementation that encounters an unknown parent digest SHOULD attempt to resolve the parent from its registry before rejecting the token, with a configurable resolution timeout.
33. All tokens in a lineage chain MUST carry the same `meta.asset_id` value. A version token carrying a different `asset_id` than its resolved parent MUST be rejected as a lineage integrity violation.
34. All tokens in a lineage chain MUST carry the same `meta.issued_by` value, unless the schema's governance rules explicitly permit issuer transfer and the transfer has been recorded in a MERGE token referencing the governance update.

5.4 Operations

The `lineage.operation` field identifies the semantic intent of the token version. Four operations are defined:

CREATE

Establishes a new logical asset in the URIB system. The CREATE operation is valid ONLY at version 1 (the genesis token). A token with `lineage.version > 1` MUST NOT carry the CREATE operation. If a token carries both `lineage.version > 1` and `lineage.operation == "CREATE"`, it MUST be rejected as a semantic violation. The CREATE operation implies that the payload represents the initial, authoritative state of the asset.

UPDATE

Mutates one or more payload fields relative to the parent token's payload. The UPDATE operation is valid for all versions greater than 1, provided the lineage chain has not been terminated by a RETIRE operation. An UPDATE token MUST NOT appear after a RETIRE token in the mainline chain. UPDATE tokens SHOULD carry a non-null `delta_ref` at Level 3 conformance. The `meta.schema_id` of an UPDATE token MAY differ from its parent's `schema_id` if the update constitutes a schema migration, in which case the UPDATE's payload MUST conform to the new schema.

MERGE

Reconciles two branches of a lineage chain back into the mainline (or a target branch). A MERGE token extends the standard CDM with an additional field: `parent_digests` (an array of exactly two digest strings), replacing the singular `parent_digest` field. Both parent digests MUST reference verified tokens from their respective branches. The version number of the MERGE token MUST be max(branchA.version, branchB.version) + 1. The MERGE token's payload MUST represent the resolved state after reconciliation, which MUST be explicitly authored (no automatic merge is performed). The MERGE operation MUST carry a non-null `delta_ref` that references a delta object documenting the reconciliation decisions.

RETIRE

Marks the logical asset as permanently end-of-life. A RETIRE token is the terminal token in a mainline lineage chain. After a RETIRE token is accepted, no further token versions MUST be accepted for the same asset on the mainline. The payload of a RETIRE token MAY be emptied to the empty object `{}` if the schema permits it, or MAY carry the final state of the asset. The RETIRE operation is final and irreversible within the URIB protocol — there is no "un-retire" operation. Client-submitted tokens claiming versions after a mainline RETIRE MUST be rejected at ingest with error code `LINEAGE_AFTER_RETIRE`.

Operation	Valid Versions	parent_digest	Terminal?	Payload Requirement
`CREATE`	1 only	MUST be null	No	Full payload per schema
`UPDATE`	>= 2	MUST be set	No	Full payload per schema (changed or unchanged fields)
`MERGE`	>= 2	MUST use `parent_digests` array	No	Full reconciled payload
`RETIRE`	>= 2	MUST be set	YES	MAY be `{}` or final state

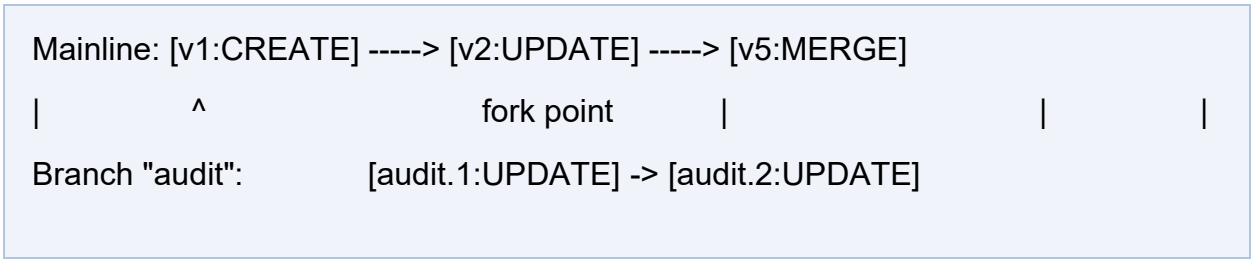
5.5 Branching Rules

Branching allows parallel evolution of an asset's state from a common ancestor, enabling use cases such as regulatory review tracks, draft/published lifecycle states, audit branches, and speculative updates pending approval.

A branch is created by issuing a new token that references a prior mainline (or branch) token's digest as its `parent_digest` and carries a non-null `lineage.branch` label. The branch label MUST be a non-empty, URL-safe ASCII string. Branch labels MUST be unique within an asset's complete lineage — no two branches of the same asset may share a label.

Branch versioning: tokens on a branch carry independent, monotonically-increasing version integers scoped to the branch. The branch's version sequence starts at 1 from the fork point. The canonical qualified version reference for a branch token is `<branch_label>.<version>` (e.g., `audit.1`, `audit.2`, `review.1`).

A branch MAY be reconciled back to the mainline (or another branch) via a MERGE token. A branch MAY also be independently RETIRED, in which case no further tokens may be issued on that branch, but the mainline and other branches are unaffected.



5.6 Delta Objects

A delta object is a canonicalized JSON document that describes the field-level diff between a token's payload and its parent's payload. Delta objects enable efficient change tracking, audit trail generation, and partial-payload synchronization without requiring consumers to diff full token payloads manually.

The normative delta object format is:

```
{   "added":    { "<field_name>": <new_value> },   "modified": {
"<field_name>": { "from": <old_value>, "to": <new_value> } },   "removed":
["<field_name>"] }
```

The rules governing delta objects are as follows:

- All three keys (`added`, `modified`, `removed`) MUST be present in every delta object, even if empty (`{}` or `[]`).
- Delta objects MUST themselves be canonicalized per Section 3 and hashed per Section 4. The resulting digest is stored in `lineage.delta_ref`.
- The delta object is stored separately from the token — it is NOT embedded inline in the `lineage` block. It is referenced only by its digest.
- The `modified` block MUST use the canonical representations of both `from` and `to` values, matching the canonical types used in the payload (NFC strings, canonical numbers, etc.).
- For MERGE tokens, the delta object describes the diff between the MERGE token's payload and the primary parent's payload, with a `"merge_source"` extension field listing the secondary parent digest.

- At Level 3 conformance (Section 10.1), implementations MUST compute and store delta objects for all UPDATE and MERGE tokens.

5.7 Lineage Integrity Verification

To perform a complete lineage integrity verification of an asset's full history, a verifier MUST execute the following procedure. The pseudocode implementation is provided in Section 9.6.

35. **Resolve all tokens.** Retrieve the complete ordered sequence of tokens for the asset, sorted by lineage.version ascending, from the URIB registry.
36. **Verify the genesis token.** Apply all genesis token rules from Section 5.2. Verify: version == 1, parent_digest == null, operation == "CREATE", branch == null, and asset_id == digest (computed).
37. **Verify each subsequent token.** For each token at version i > 1:
    - Verify that lineage.version == previous.lineage.version + 1.
    - Verify that lineage.parent_digest == previous.digest (exact string equality).
    - Verify that meta.asset_id == genesis.digest.
    - Verify the token's digest independently per Section 4.3.
38. **Verify operation sequence validity.** Check that no token version appears after a token with operation == "RETIRE" on the mainline. Check that CREATE appears only at version 1.
39. **Verify branch integrity (if applicable).** For each branch, apply the same sequential verification within the branch's version sequence. Verify MERGE tokens reference valid digests from both contributing branches.
40. **Report results.** Return a pass/fail result with the verified chain length, the final state token, and any integrity violations encountered.

6 Reversible Mapping Guarantees

6.1 Definition of Reversibility

A reversible mapping in the URIB context is a bijective (one-to-one, onto) transformation pair between a token's canonical payload and its corresponding domain-native source representation. Formally, for a domain D with domain-native representation space N and canonical representation space C:

```
canonicalize: N → C    // Transform domain-native to canonical restore:
C → N    // Transform canonical to domain-native  Such that:
restore(canonicalize(n)) ≡ n   ∀ n ∈ N  (semantic equivalence)
canonicalize(restore(c)) = c   ∀ c ∈ C  (exact byte equivalence)
```

The first condition (semantic equivalence) means that the round-trip from domain-native through canonical and back to domain-native produces a result that is semantically identical to the original — no data is lost, transformed, or approximated in a way that would affect downstream processing in the source system. The second condition (exact byte equivalence) means that the reverse round-trip from canonical through domain-native and back to canonical produces a bit-perfect reproduction of the canonical form.

A mapping that satisfies both conditions is declared **fully reversible**. A mapping that cannot satisfy the first condition due to inherent source-system limitations (e.g., source system truncates decimal precision) MUST be declared partially lossy in the schema as described in Section 6.2.

6.2 Mapping Contract

The following requirements govern the reversible mapping contract for every URIB token schema:

41. Every URIB-compatible schema MUST document a `restore` function for its domain. The restore function MUST be expressed in the schema's `x-urib-mapping-table` extension (see Section 7.3) and MUST specify, for each mapped field: the source field name, any transformation applied, and whether the transformation is lossless.
42. The restore function MUST reconstruct the domain-native representation from the canonical payload without data loss, for all fields declared as `"lossless": true` in the mapping table.

43. Transformations that are inherently lossy — such as precision reduction (e.g., truncating a 6-decimal-place source value to 4 decimal places), field omission (e.g., dropping metadata not relevant to the canonical model), or type narrowing (e.g., mapping a variable-length string to a fixed-length string) — MUST be explicitly declared in the mapping table with `"lossless": false`.
44. Schemas containing any `"lossless": false` transformation MUST declare `"x-urib-lossless": false` at the schema root level. Such tokens MUST NOT be used in contexts where full round-trip fidelity is required (e.g., regulatory reporting, legal documents, financial settlement).
45. Type coercions — transformations between different primitive types (e.g., a source-system string "195.42" coerced to a JSON number `195.42`) — MUST be documented in the mapping table even if semantically lossless. The restore function MUST perform the inverse coercion.

6.3 Primitive Type Mapping Table

The following table defines the normative canonical type mappings between JSON canonical types and common domain-native types. For each mapping, the directionality, normalization requirements, and losslessness assessment are specified.

Canonical Type	Domain-Native Types	Lossless?	Normalization / Notes
`string` (UTF-8 NFC)	VARCHAR, TEXT, NVARCHAR, any string type	Conditional	Lossless if source supports Unicode. Lossy if source is ASCII-only and source contains non-ASCII characters. NFC normalization MUST be applied.
`number` (JSON integer)	INT, BIGINT, SMALLINT, byte, short, long, uint	Yes (within range)	JSON numbers support arbitrary precision. Source integer overflow relative to JSON range MUST be declared lossy. Unsigned integers MUST be represented as positive JSON numbers.
`number` (JSON decimal)	FLOAT, DOUBLE, DECIMAL, NUMERIC, fixed-point	Conditional	Lossless if source precision is preserved in the canonical number. Lossy if source carries more precision than the canonical form can represent. Mapping table MUST specify scale and precision.

Canonical Type	Domain-Native Types	Lossless?	Normalization / Notes
`boolean`	BOOL, BIT, TINYINT(1), Y/N char, 0/1 int	Yes	Any truthy/falsy source representation MUST be mapped to JSON `true` or `false`. The inverse mapping MUST reconstruct the source-specific representation per the mapping table declaration.
`null`	NULL, undefined, empty string, zero, blank	Conditional	The specific source-system null representation (e.g., empty string vs. SQL NULL vs. sentinel zero) MUST be declared in the mapping table if round-trip fidelity is required. Conflating distinct null-like values into a single JSON null is lossy.
`string` (ISO 8601 UTC)	DATE, DATETIME, TIMESTAMP, epoch integer	Conditional	All datetime values MUST be converted to UTC and expressed in ISO 8601 with millisecond precision and Z suffix. Original timezone offset MUST be preserved in mapping metadata if round-trip fidelity is required. See Section 6.4.
`array` (JSON)	LIST, SET, ARRAY, collection, repeated field	Conditional	Element order MUST be preserved unless schema declares order-insensitive. Sets (unordered collections) are lossy when mapped to ordered arrays without a canonical element sort order declaration.
`object` (JSON)	STRUCT, RECORD, MAP, nested entity	Yes	Nested objects follow the same canonicalization rules recursively. Key ordering is determined by the canonical serialization algorithm. The mapping table MUST document any field renaming.

6.4 Timezone Normalization

All datetime values in the canonical form of a URIB token MUST be expressed in UTC (Coordinated Universal Time) using the ISO 8601 format with millisecond precision and the `Z` suffix. This normalization ensures that datetime comparisons and ordering operations produce consistent results across implementations without timezone-awareness requirements.

The normalization procedure for datetime values is:

46. Parse the source-system datetime value, including its timezone offset if present.
47. Convert the value to UTC by applying the timezone offset.
48. Express the UTC value as `YYYY-MM-DDTHH:mm:ss.sssZ`, padding milliseconds to three decimal places.

49. If the source datetime carried a non-UTC timezone offset and the application requires round-trip restoration of the original timezone, the original offset MUST be preserved in the schema's mapping metadata field `x-urib-tz-preserve: true`, and a companion field (e.g., `original_tz_offset`) MUST be declared in the schema to carry the offset string (e.g., `"-05:00"`).

Implementations MUST NOT silently discard timezone information. If a source datetime carries a timezone offset and the schema does not declare `x-urib-tz-preserve: true`, the implementation MUST treat the conversion as lossy and the schema MUST declare `x-urib-lossless: false`.

6.5 Identity-Preserving Fields

Certain fields in a token payload carry values that are authoritative identifiers in the source domain — values that MUST be reproduced with exact character-level fidelity and MUST NOT be subject to any transformation, normalization beyond the mandatory UTF-8 NFC step, or truncation. Examples include: ISIN codes, CUSIPs, national identification numbers, regulatory identifiers, and cryptographic keys.

Fields declared as identity-preserving are specified in the schema's `x-urib-identity-fields` extension array (see Section 7.3). The following constraints apply to all identity-preserving fields:

- Whitespace trimming MUST NOT be applied to the field value in either the canonicalize or restore direction, unless the schema explicitly permits it.
- Case normalization (uppercasing, lowercasing) MUST NOT be applied to the field value, unless the schema explicitly permits it and the original case information is not required for round-trip fidelity.
- Encoding transformation beyond UTF-8 NFC normalization MUST NOT be applied. If the source system encodes the identifier in a non-Unicode encoding, the conversion to Unicode MUST be performed exactly once at canonicalization time and the mapping documented.
- Any schema-defined pattern constraint (e.g., `"pattern": "^[A-Z]{2}[A-Z0-9]{10}$"` for ISIN) MUST be satisfied by the canonical value. If the source value does not satisfy the pattern, the token MUST be rejected at schema validation time.

7 Schema-Binding Requirements

7.1 Overview

Every URIB token is bound to exactly one versioned JSON Schema that governs the structure, types, constraints, and mapping metadata of its payload. Schema binding is the mechanism by which the URIB platform achieves type safety, semantic clarity, and reversibility across domain boundaries and over time.

Schema binding serves four distinct functions: (1) **field validation** — ensuring the payload contains exactly the fields declared in the schema, with the correct types and constraint satisfaction; (2) **type enforcement** — providing precise type information that enables canonical serialization to apply the correct numeric and string normalization rules; (3) **reversibility metadata** — supplying the mapping table that the restore function uses to reconstruct domain-native representations; and (4) **versioning governance** — providing the basis for determining whether a schema change is breaking or non-breaking, and therefore whether existing tokens remain valid under the new schema version.

A token MUST be bound to exactly one schema at any given time. A token carrying a `schema_id` that references a non-existent, deprecated (and removed), or structurally incompatible schema MUST be rejected.

7.2 Schema Identifier Format

The `meta.schema_id` field carries a colon-delimited three-part identifier that unambiguously references a specific version of a URIB-compatible JSON Schema:

```
<namespace>:<schema_name>:<semver>
```

The three components are defined as follows:

- **`namespace`**: A dot-delimited domain hierarchy string that organizes schemas by their domain context. The namespace MUST match the pattern `[a-z][a-z0-9]*(\.[a-z][a-z0-9]*)+`. Reserved namespaces are defined in Appendix A. Custom namespaces MUST be registered in the URIB schema registry before use.

- **`schema_name`**: The name of the schema within its namespace. MUST be in `snake_case` (lowercase letters, digits, and underscores; no hyphens, no spaces). MUST match the pattern `[a-z][a-z0-9_]*`.
- **`semver`**: The version of the schema, following Semantic Versioning 2.0.0. MUST be in the form `MAJOR.MINOR.PATCH` (e.g., `1.2.0`). Pre-release identifiers and build metadata MUST NOT be used in production schema identifiers.

Example valid `schema_id` values:

- `finance.instrument:equity_token:1.2.0`
- `health.record:patient_encounter:2.0.1`
- `identity.credential:verifiable_id:1.0.0`
- `supply.asset:shipment_manifest:3.1.0`
- `iot.telemetry:device_reading:1.0.0`

7.3 Schema Structure

Every URIB-compatible JSON Schema MUST be a valid JSON Schema draft-07 document and MUST include the following standard fields and URIB extension fields:

Mandatory standard JSON Schema fields:

- `"$schema"`: MUST be `"http://json-schema.org/draft-07/schema#"`.
- `"$id"`: MUST be the fully-qualified schema URI in the form `urib:<schema_id>`.
- `"type": "object"`: The payload is always a JSON object.
- `"additionalProperties": false`: No fields beyond those declared in `properties` are permitted.
- `"properties"`: Declaration of all payload fields with their types and constraints.
- `"required"`: Array of field names that MUST be present in every token's payload.

Mandatory URIB extension fields:

- `"x-urib-domain"`: A string matching one of the reserved domain identifiers in Appendix A, or a registered custom domain. MUST match `meta.domain` in all tokens using this schema.
- `"x-urib-lossless"`: Boolean. `true` if all mappings in this schema are lossless; `false` if any mapping is lossy. Defaults to `true` if omitted.
- `"x-urib-identity-fields"`: An array of field name strings identifying payload fields that are identity-preserving (see Section 6.5). MAY be an empty array.
- `"x-urib-mapping-table"`: An object mapping canonical field names to their domain-native mapping metadata. Each entry MUST specify: `source_field` (the name in the domain-native system), `transform` (the transformation applied, or `"none"`), and `lossless` (boolean).

The following is a complete, annotated example schema for a financial equity instrument token:

```
{   "$schema": "http://json-schema.org/draft-07/schema#",   "$id":
"urib:finance.instrument:equity_token:1.0.0",   "title":   "Equity Token",
"description": "Canonical token schema for publicly-traded equity
instruments.",    "x-urib-domain":         "finance.instrument",   "x-urib-
lossless":       true,   "x-urib-identity-fields": ["isin", "ticker"],
"type": "object",   "additionalProperties": false,    "required": ["isin",
"ticker", "name", "currency", "exchange", "price_usd"],    "properties": {
"currency":       { "type": "string",  "pattern": "^[A-Z]{3}$" },
"exchange":       { "type": "string" },     "isin":           { "type":
"string",  "pattern": "^[A-Z]{2}[A-Z0-9]{10}$" },     "market_cap_usd": {
"type": ["number", "null"] },     "name":           { "type": "string" },
"price_usd":      { "type": "number",  "minimum": 0 },     "tags":
{ "type": "array",   "items": { "type": "string" } },     "ticker":         {
"type": "string",  "maxLength": 10 }   },    "x-urib-mapping-table": {
"currency":       { "source_field": "currency_code",  "transform": "none",
"lossless": true },     "exchange":       { "source_field": "exchange_mic",
"transform": "none",          "lossless": true },     "isin":           {
"source_field": "isin",           "transform": "none",          "lossless":
true },     "market_cap_usd": { "source_field": "mkt_cap",
"transform": "none",          "lossless": true },     "name":           {
"source_field": "security_name",  "transform": "trim",          "lossless":
true },     "price_usd":      { "source_field": "last_price",
"transform": "divide_by_100","lossless": true },     "tags":           {
"source_field": "classifications", "transform": "none",         "lossless":
true },     "ticker":         { "source_field": "symbol",
"transform": "uppercase",    "lossless": true }   } }
```

7.4 Schema Versioning and Compatibility

Schema versioning follows Semantic Versioning 2.0.0 semantics adapted to the URIB context. The compatibility rules are as follows:

Patch version increments (e.g., 1.0.0 → 1.0.1): Reserved for corrections to schema documentation, description fields, or `title` text that do not affect validation behavior. No field additions, removals, or type changes. Existing tokens remain valid without modification.

Minor version increments (e.g., 1.0.0 → 1.1.0): Additive-only changes. New **optional** fields MAY be added to `properties`. New entries MAY be added to `x-urib-mapping-table`. No existing field MAY be removed, renamed, or have its type changed or constraint tightened. Existing tokens produced under version 1.0.0 MUST remain valid under version 1.1.0. Minor version bumps MUST NOT add new required fields.

Major version increments (e.g., 1.0.0 → 2.0.0): Breaking changes. Any of the following constitute a major version increment: removal of an existing field; change of an existing field's type; tightening of an existing constraint (e.g., reducing `maxLength`); adding a new required field; renaming a field; changing the `x-urib-domain` value. Existing tokens produced under version 1.x.x are NOT automatically valid under version 2.0.0. Migration requires issuing UPDATE tokens that carry the new `schema_id` and whose payloads conform to the new schema version.

Implementations MUST reject tokens whose payload violates the schema referenced by their `meta.schema_id`, regardless of whether a newer schema version would accept the payload.

7.5 Schema Registry Requirements

A conformant URIB implementation at Level 2 or above (Section 10.1) MUST provide or integrate with a schema registry that satisfies the following requirements:

50. **Addressability by `schema_id`:** The registry MUST be capable of resolving any valid `schema_id` string to the corresponding JSON Schema document. Resolution MUST be deterministic: the same `schema_id` MUST always resolve to the same schema document.
51. **Version resolution:** The registry MUST support resolution of the `latest` alias to the currently published highest non-deprecated version of a schema within a given namespace and schema name. The alias MUST NOT be used in token `schema_id` fields — tokens MUST always carry a fully-resolved version string.
52. **Immutability:** Once a schema version has been published to the registry, the content of that schema version MUST NOT be mutated. If a correction is required, a new patch version MUST be published. The registry MUST enforce this constraint with write-once semantics for published versions.
53. **Deprecation support:** The registry MUST support marking schema versions as deprecated. Deprecated schemas MUST remain resolvable — they MUST NOT be removed. Implementations SHOULD emit warnings when validating tokens against deprecated schema versions.
54. **Secure retrieval:** Schema documents MUST be retrievable over a verified channel. For HTTP retrieval, TLS MUST be used. Content-addressable storage (e.g., retrieving by the SHA3-256 hash of the schema content) is RECOMMENDED as an additional integrity measure.

7.6 Payload Validation Sequence

A conformant URIB implementation MUST execute the following payload validation sequence for every received token, after digest verification (Section 4.3) has passed:

55. **Extract `schema_id`.** Read the `meta.schema_id` field from the verified token.
56. **Resolve the schema.** Query the schema registry with the extracted `schema_id`. If the schema is not found, reject the token with error code `SCHEMA_NOT_FOUND`. If the schema is deprecated, log a warning and proceed (unless the implementation policy requires rejection of deprecated-schema tokens, in which case reject with code `SCHEMA_DEPRECATED`).
57. **Validate `meta.domain` consistency.** Verify that `meta.domain` equals the schema's `x-urib-domain` value. Reject with `SCHEMA_VIOLATION` if they differ.
58. **Validate the payload.** Execute a JSON Schema draft-07 conformant validator against the token's `payload` block using the resolved schema. The validator MUST enforce: required field presence, type constraints, pattern constraints, numeric range constraints, array item type constraints, and `additionalProperties: false`.
59. **Handle validation failures.** If validation fails for any reason, reject the token with error code `SCHEMA_VIOLATION`. The rejection response MUST include: the `token_id`, the `schema_id`, and the full list of validation errors from the JSON Schema validator. MUST NOT silently accept a token that fails schema validation.
60. **Confirm identity field integrity.** For each field listed in the schema's `x-urib-identity-fields`, verify that the field value satisfies any declared pattern constraint and has not been subject to prohibited normalization. Reject with `SCHEMA_VIOLATION` if any identity field fails this check.
61. **Proceed to business logic.** Only after steps 1–6 complete without error may the token's payload be passed to application-layer business logic.

Normative — Validation Order

The mandatory processing order for any received URIB token is: (1) CDM structure check, (2) digest verification (Section 4.3), (3) schema resolution and payload validation (this section). Implementing these steps in any other order is non-conformant. In particular, business logic MUST NOT be invoked before both digest verification and schema validation have passed.

---

8 Complete End-to-End Examples

Informative — Example Digests

The digest values shown in these examples are illustrative 64-character hex strings. They are not computed values. In a real implementation, digests MUST be computed by applying the canonical serialization algorithm (Section 3.2) to the token with `"digest": null` and then applying SHA3-256 to the resulting byte sequence. The asset_id in the genesis example equals the genesis token's own digest, as required by Section 5.2.

8.1 Example 1: Genesis Token (Finance Domain)

The following is a complete, valid URIB genesis token for an equity instrument. All fields are populated, the digest is computed, and the schema_id references the equity token schema defined in Section 7.3. The payload keys are in Unicode code point order as required by the canonical serialization algorithm.

```
{   "digest":
"a3f8c2d1e4b7096f3a2c8d5e1f4b7a9c2d3e6f0b1a4c7d2e5f8a0b3c6d9e2f5",
"lineage": {     "branch":        null,     "delta_ref":     null,
"operation":     "CREATE",     "parent_digest": null,     "version":       1
},    "meta": {     "asset_id":
"a3f8c2d1e4b7096f3a2c8d5e1f4b7a9c2d3e6f0b1a4c7d2e5f8a0b3c6d9e2f5",
"created_at":   "2026-07-09T00:00:00.000Z",     "domain":
"finance.instrument",     "issued_by":    "urn:urib:issuer:base44-platform",
"schema_id":    "finance.instrument:equity_token:1.0.0",     "token_id":
"018f2e3a-5b1c-7d2e-9f4a-0b3c8d1e6f2a",     "urib_version": "1.0.0"   },
"payload": {     "currency":       "USD",     "exchange":       "NASDAQ",
"isin":           "US0378331005",     "market_cap_usd": 3120000000000,
"name":           "Apple Inc.",     "price_usd":      195.42,     "tags":
["technology", "large-cap", "sp500"],     "ticker":         "AAPL"   } }
```

Field annotations:

- `digest`: Top-level field sorts before `lineage`, `meta`, and `payload` in Unicode code point order. Equals `meta.asset_id` because this is the genesis token.
- `lineage.version = 1`, `lineage.parent_digest = null`, `lineage.operation = "CREATE"`: All genesis token requirements satisfied.
- `meta.asset_id`: Equals the token's own digest — the defining self-referential property of a genesis token.
- `meta.created_at`: ISO 8601 UTC with millisecond precision and Z suffix.
- `payload` keys: `currency`, `exchange`, `isin`, `market_cap_usd`, `name`, `price_usd`, `tags`, `ticker` — sorted in Unicode code point order.
- `market_cap_usd = 3120000000000`: Integer, no decimal point (Step 5 compliance).
- `price_usd = 195.42`: Shortest representation, no trailing zeros (Step 5 compliance).

8.2 Example 2: Update Token (Price Change)

The following token represents a version 2 update to the asset established in Example 8.1. The price has changed from 195.42 to 201.87. The `asset_id` is unchanged. The `parent_digest` references the genesis token's digest. A delta object has been computed and its digest is stored in `delta_ref`.


```
{   "digest":
"7b2c9e4f1a8d3c6e0b5f2a9d4c7e1f3a8b2d5e9c0f4a7b1d3e6c8f0a2b5d9e1c",
"lineage": {     "branch":        null,     "delta_ref":
"f1a4c7d2e5b8093f2a1c9d6e3f7b0a4c2d5e8f1b3a6c9d0e2f5b8a1c4d7e0f3",
"operation":     "UPDATE",     "parent_digest":
"a3f8c2d1e4b7096f3a2c8d5e1f4b7a9c2d3e6f0b1a4c7d2e5f8a0b3c6d9e2f5",
"version":       2   },    "meta": {     "asset_id":
"a3f8c2d1e4b7096f3a2c8d5e1f4b7a9c2d3e6f0b1a4c7d2e5f8a0b3c6d9e2f5",
"created_at":   "2026-07-09T00:06:00.000Z",     "domain":
"finance.instrument",     "issued_by":    "urn:urib:issuer:base44-platform",
"schema_id":    "finance.instrument:equity_token:1.0.0",     "token_id":
"018f2e4b-1a2c-7d3e-8f5b-0c4d9e2f7a1b",     "urib_version": "1.0.0"   },
"payload": {     "currency":       "USD",     "exchange":       "NASDAQ",
"isin":           "US0378331005",     "market_cap_usd": 3204000000000,
"name":           "Apple Inc.",     "price_usd":      201.87,     "tags":
["technology", "large-cap", "sp500"],     "ticker":         "AAPL"   } }
```


Key differences from Example 8.1:

- `lineage.version = 2`: Incremented by exactly 1 from the genesis token's version of 1.
- `lineage.operation = "UPDATE"`: Reflects a payload mutation operation.
- `lineage.parent_digest`: Set to the digest of the v1 genesis token.
- `lineage.delta_ref`: Non-null — references a separately stored and canonicalized delta object that documents the `price_usd` change from 195.42 to 201.87 and the `market_cap_usd` change.
- `meta.asset_id`: Unchanged — the permanent Asset ID from the genesis token.
- `meta.token_id`: A new, unique UUIDv7 for this specific token instance.
- `payload.price_usd`: Updated from 195.42 to 201.87.
- `payload.market_cap_usd`: Incidentally updated as well. The delta object captures both changes.

8.3 Example 3: RETIRE Token

The following token retires the asset established in Example 8.1 and updated in Example 8.2. It is the terminal token in this asset's mainline lineage chain. The payload is emptied to `{}`. After this token is accepted, no further versions may be submitted for this asset's mainline.


```
{   "digest":
"c4e8f2b1a6d9034e7c1f5b3a8e2d6c0f4b9a3e7d1c5f0b2a9e6d4c8f3b0a5e9d",
"lineage": {     "branch":        null,     "delta_ref":     null,
"operation":     "RETIRE",     "parent_digest":
"7b2c9e4f1a8d3c6e0b5f2a9d4c7e1f3a8b2d5e9c0f4a7b1d3e6c8f0a2b5d9e1c",
"version":       3   },    "meta": {     "asset_id":
"a3f8c2d1e4b7096f3a2c8d5e1f4b7a9c2d3e6f0b1a4c7d2e5f8a0b3c6d9e2f5",
"created_at":   "2026-07-09T00:06:30.000Z",     "domain":
"finance.instrument",     "issued_by":    "urn:urib:issuer:base44-platform",
"schema_id":    "finance.instrument:equity_token:1.0.0",     "token_id":
"018f2e5c-2b3d-7e4f-9a6c-1d5e0f3b8c2d",     "urib_version": "1.0.0"   },
"payload": {} }
```


Key characteristics of this RETIRE token:

- `lineage.operation = "RETIRE"`: Marks this as a terminal token.
- `lineage.version = 3`: Correctly incremented by 1 from version 2.
- `lineage.parent_digest`: Set to the v2 token's digest.
- `payload = {}`: Emptied, as permitted for RETIRE tokens when the schema allows it.
- `lineage.delta_ref = null`: No delta is required for a RETIRE token with empty payload.
- After this token is accepted, any attempt to submit a token with `asset_id = "a3f8c..."` and `version >= 4` MUST be rejected with `LINEAGE_AFTER_RETIRE`.

8.4 Example 4: Invalid Token (Rejected)

The following token illustrates three distinct violations of this specification. A conformant implementation MUST reject this token. Each violation is annotated.

```
{   "meta": {     "urib_version": "1.0.0",     "token_id":     "018f2e3a-
5b1c-7d2e-9f4a-0b3c8d1e6f2a",     "asset_id":
"a3f8c2d1e4b7096f3a2c8d5e1f4b7a9c2d3e6f0b1a4c7d2e5f8a0b3c6d9e2f5",
"schema_id":    "finance.instrument:equity_token:1.0.0",     "domain":
"finance.instrument",     "created_at":   "2026-07-09T00:00:00.000Z",
"issued_by":    "urn:urib:issuer:base44-platform"   },   "payload": {
"ticker":         "AAPL",     "isin":           "US0378331005",     "name":
"Apple Inc.",     "currency":       "USD",     "exchange":       "NASDAQ",
"price_usd":      195.42,     "market_cap_usd": 3120000000000,     "tags":
["technology", "large-cap", "sp500"]   },   "lineage": {     "version":
3,     "parent_digest":
"a3f8c2d1e4b7096f3a2c8d5e1f4b7a9c2d3e6f0b1a4c7d2e5f8a0b3c6d9e2f5",
"branch":        null,     "operation":     "UPDATE",     "delta_ref":
null   },   "digest":
"0000000000000000000000000000000000000000000000000000000000000000" }
```

Violations identified:

#	Violation	Location	Applicable Rule
1	**Payload keys not in Unicode code point order.** The payload keys appear in the order: `ticker`, `isin`, `name`, `currency`, `exchange`, `price_usd`, `market_cap_usd`, `tags`. The correct canonical order is: `currency`, `exchange`, `isin`, `market_cap_usd`, `name`, `price_usd`, `tags`, `ticker`.	`payload` block	Section 3.2 Step 3 — Keys MUST be sorted in Unicode code point order at every level of nesting.
2	**Digest does not match computed hash.** The `digest` field contains 64 zeros (`0000...0000`), which is clearly not the SHA3-256 hash of the token's canonical serialization. Any conforming verifier will compute a different digest and reject the token.	`digest` field	Section 4.3 — The computed digest MUST match the claimed digest byte-for-byte. Token MUST be rejected if they differ.
3	**Version number gap.** The token claims `lineage.version = 3` while its `parent_digest` references the genesis token (version 1). This implies a jump from version 1 to version 3, skipping version 2. Version gaps are explicitly prohibited.	`lineage.version`	Section 5.3 Rule 1 — Each token MUST increment `lineage.version` by exactly 1 relative to its parent. Gaps are prohibited.

9 Developer Templates

Informative — Template Usage

The templates in this section are ready-to-use starting points for common token issuance scenarios. Replace all `<PLACEHOLDER>` values with actual values before computing the digest. The digest MUST be computed last, as specified in Section 4.2, after all other fields are finalized.

9.1 Genesis Token Template

```
{   // TOP-LEVEL STRUCTURE — exactly four keys, in Unicode code point order
"digest": "<COMPUTE_LAST: SHA3-256 of canonical serialization with
digest=null>",    "lineage": {     "branch":        null,          // Always
null at genesis     "delta_ref":     null,          // Always null at genesis
(no parent to diff)     "operation":     "CREATE",      // Always CREATE at
genesis     "parent_digest": null,          // Always null at genesis
"version":       1              // Always 1 at genesis   },    "meta": {
"asset_id":     "<SAME AS DIGEST: copy after computing digest above>",
"created_at":   "<ISO 8601 UTC, e.g. 2026-07-09T00:00:00.000Z>",
"domain":       "<DOMAIN: e.g. finance.instrument — see Appendix A>",
"issued_by":    "urn:urib:issuer:<YOUR_SYSTEM_IDENTIFIER>",     "schema_id":
"<NAMESPACE>:<SCHEMA_NAME>:<SEMVER>",     "token_id":     "<UUIDv7: time-
ordered, globally unique per token instance>",     "urib_version": "1.0.0"
},    "payload": {     // Replace with domain-specific fields governed by
schema_id     // All keys MUST be in Unicode code point order     // All
strings MUST be UTF-8 NFC-normalized     "<field_1>": "<value_1>",
"<field_2>": <value_2>   } }
```

9.2 Update Token Template

```
{   "digest": "<COMPUTE_LAST: SHA3-256 of canonical serialization with
digest=null>",    "lineage": {     "branch":        null,          // null
for mainline updates     "delta_ref":     "<DIGEST OF SEPARATELY STORED DELTA
OBJECT, or null>",     "operation":     "UPDATE",     "parent_digest": "<64-
char hex digest of the immediately preceding token>",     "version":
<PARENT_VERSION + 1>  // Must be exactly parent + 1   },    "meta": {
"asset_id":     "<GENESIS TOKEN DIGEST: unchanged across all versions>",
"created_at":   "<ISO 8601 UTC with millisecond precision, Z suffix>",
"domain":       "<SAME DOMAIN AS GENESIS TOKEN>",     "issued_by":
"urn:urib:issuer:<YOUR_SYSTEM_IDENTIFIER>",     "schema_id":    "<SCHEMA_ID:
may differ from parent if migrating schemas>",     "token_id":     "<NEW
UUIDv7: different from all prior token_ids>",     "urib_version": "1.0.0"
},    "payload": {     // Full payload — all schema-required fields present,
not just changed fields     // Changed fields carry the new values     //
Unchanged fields carry the same values as the parent token     "<field_1>":
"<new_or_unchanged_value_1>",     "<field_2>": <new_or_unchanged_value_2>   }
}
```

9.3 Retire Token Template

```
{   "digest": "<COMPUTE_LAST: SHA3-256 of canonical serialization with
digest=null>",    "lineage": {     "branch":        null,     "delta_ref":
null,          // Typically null for RETIRE     "operation":     "RETIRE",
// Terminal operation — no further versions permitted     "parent_digest":
"<64-char hex digest of the preceding token>",     "version":
<PARENT_VERSION + 1>   },    "meta": {     "asset_id":     "<GENESIS TOKEN
DIGEST: unchanged>",     "created_at":   "<ISO 8601 UTC with millisecond
precision, Z suffix>",     "domain":       "<DOMAIN>",     "issued_by":
"urn:urib:issuer:<YOUR_SYSTEM_IDENTIFIER>",     "schema_id":
"<SCHEMA_ID>",     "token_id":     "<NEW UUIDv7>",     "urib_version":
"1.0.0"   },    "payload": {}  // MAY be empty object for RETIRE tokens, or
final asset state }
```

9.4 Schema Template

```
{   "$schema": "http://json-schema.org/draft-07/schema#",   "$id":
"urib:<namespace>:<schema_name>:<semver>",   "title":   "<Human-Readable
Schema Title>",   "description": "<Description of the domain entity this
schema represents.>",    "x-urib-domain":          "<namespace (matches
meta.domain)>",   "x-urib-lossless":         true,   "x-urib-identity-
fields": ["<field_that_is_identity_preserving>"],    "type": "object",
"additionalProperties": false,    "required": [     "<required_field_1>",
"<required_field_2>"   ],    "properties": {     "<field_1>": {       "type":
"string",       "description": "<Field description.>"     },     "<field_2>":
{       "type": "number",       "minimum": 0,       "description": "<Numeric
field, non-negative.>"     },     "<nullable_field>": {       "type":
["string", "null"],       "description": "<Field that may be null.>"     }
},    "x-urib-mapping-table": {     "<field_1>": {       "source_field":
"<source_system_field_name>",       "transform":    "none",       "lossless":
true     },     "<field_2>": {       "source_field":
"<source_system_field_name>",       "transform":    "<e.g. divide_by_100,
uppercase, trim>",       "lossless":     true     }   } }
```

9.5 Digest Computation Pseudocode

```
// ─── ENTRY POINT ──────────────────────────────────────────────────────────── function
computeDigest(token):     working = deepCopy(token)     working["digest"] =
null          // Step 8: substitute digest placeholder     serialized =
canonicalSerialize(working)   // Apply Section 3 rules     hashBytes =
sha3_256(serialized)           // FIPS 202, SHA3-256     return
hexEncode(hashBytes).toLowerCase()  // 64-char lowercase hex  // ─── CANONICAL SERIALIZER ──────────────────────────────────────────────────────
function canonicalSerialize(value):     if isObject(value):         // Step
4: NFC-normalize all keys before sorting         normalizedKeys =
[nfcNormalize(k) for k in value.keys()]         // Step 3: Sort keys by
Unicode code point order (ascending)         sortedKeys =
sort(normalizedKeys, by=UNICODE_CODE_POINT_ORDER)         pairs = []
for k in sortedKeys:             serializedKey = jsonEncodeString(k)
serializedVal = canonicalSerialize(value[k])
pairs.append(serializedKey + ":" + serializedVal)         return "{" +
join(pairs, ",") + "}"      if isArray(value):         // Step 7: Preserve
array element order (order-sensitive by default)         elements =
[canonicalSerialize(v) for v in value]         return "[" + join(elements,
",") + "]"      if isString(value):         // Step 4: NFC-normalize string
values         normalized = nfcNormalize(value)         return
jsonEncodeString(normalized)  // RFC 8259 string encoding      if
isNumber(value):         return canonicalNumber(value)        // See
canonicalNumber() below      if isBoolean(value):         return "true" if
value else "false"  // Step 6: lowercase literals      if isNull(value):
return "null"                          // Step 6: lowercase literal      raise
TypeError("Unsupported value type: " + typeof(value))  // ─── CANONICAL
NUMBER FORMATTING ─────────────────────────────────────────────────── function
canonicalNumber(n):     if isInteger(n):         return str(int(n))
// No decimal point for integers      absVal = abs(n)     if absVal >= 1e20
or (absVal > 0 and absVal < 1e-6):         return scientificNotation(n)
// Scientific notation for extremes      // Shortest decimal representation,
no trailing zeros     s = formatDecimal(n, stripTrailingZeros=true)
return s  // ─── DIGEST VERIFICATION ──────────────────────────────────────────
──────────────────────────────────────────────────────────── function
verifyDigest(token):     claimedDigest = token["digest"]           // Step 1:
extract claimed digest     working = deepCopy(token)     working["digest"] =
null                  // Step 2: substitute placeholder     serialized =
canonicalSerialize(working)  // Step 3: re-serialize     computed =
sha3_256(serialized)     computedHex = hexEncode(computed).toLowerCase()
// Step 6: constant-time comparison (avoid timing side channels)     return
constantTimeEquals(computedHex, claimedDigest)
```

9.6 Lineage Verification Pseudocode

```
// tokens: ordered list of token objects, genesis first, latest last function
verifyLineageChain(tokens):     if len(tokens) == 0:         raise
LineageError("Empty lineage chain")      // ─── Verify genesis token
──────────────────────────────────────────     genesis = tokens[0]
assert genesis.lineage.version == 1,         "Genesis token must have version
1"      assert genesis.lineage.parent_digest == null,         "Genesis token
must have null parent_digest"      assert genesis.lineage.operation ==
"CREATE",         "Genesis token must have operation CREATE"      assert
genesis.lineage.branch == null,         "Genesis token must have null branch"
assert verifyDigest(genesis),         "Genesis token digest verification
failed"      assert genesis.meta.asset_id == genesis.digest,         "Genesis
token asset_id must equal its own digest"      // ─── Verify subsequent
tokens ──────────────────────────────────────────     retireEncountered =
false      for i in range(1, len(tokens)):         current  = tokens[i]
previous = tokens[i - 1]          // Reject versions after RETIRE         if
retireEncountered:             raise LineageError(                 "Token at
index " + i + " appears after RETIRE operation"             )          //
Verify version increment         assert current.lineage.version ==
previous.lineage.version + 1,             "Version gap detected at index " +
i +             ": expected " + (previous.lineage.version + 1) +
", got " + current.lineage.version          // Verify parent digest linkage
assert current.lineage.parent_digest == previous.digest,             "Parent
digest mismatch at version " + current.lineage.version          // Verify
asset ID consistency         assert current.meta.asset_id == genesis.digest,
"asset_id mismatch at version " + current.lineage.version          // Verify
current token's own digest         assert verifyDigest(current),
"Digest verification failed for version " + current.lineage.version
// Check if this is a RETIRE token         if current.lineage.operation ==
"RETIRE":             retireEncountered = true      return {         "valid":
true,         "chain_length":  len(tokens),         "asset_id":
genesis.digest,         "final_version": tokens[-1].lineage.version,
"retired":       retireEncountered     }
```

10 Conformance

10.1 Conformance Levels

Three conformance levels are defined. Higher levels are cumulative — a Level 2 implementation MUST satisfy all Level 1 requirements in addition to Level 2-specific requirements. A Level 3 implementation MUST satisfy all Level 1 and Level 2 requirements.

Level 1 — Basic Conformance

A Level 1 conformant implementation MUST implement all of the following:

- The Canonical Document Model (CDM) as defined in Section 2, including all four top-level blocks and all mandatory fields therein.
- The canonical serialization algorithm as defined in Section 3.2, including all nine steps in the specified order.
- SHA3-256 digest computation as defined in Section 4.2.
- Digest verification on receipt of any token as defined in Section 4.3.
- Rejection of tokens with invalid digests, with mandatory logging per Section 4.3 Step 7.
- Genesis token validation including the self-referential `asset_id` check per Section 5.2.
- Rejection of all prohibited serialization patterns as listed in Section 3.4.

Level 2 — Full Conformance

A Level 2 conformant implementation MUST implement all Level 1 requirements plus all of the following:

- Schema-binding and payload validation as defined in Section 7, including the full payload validation sequence in Section 7.6.
- A conformant schema registry satisfying all requirements in Section 7.5.
- Lineage chain verification as defined in Section 5.7, including version gap detection, parent digest linkage verification, and RETIRE finality enforcement.
- Reversible mapping documentation for all registered schemas, satisfying the mapping contract in Section 6.2.
- Asset ID derivation and cross-version lookup as defined in Section 4.4.
- All four operation types (CREATE, UPDATE, MERGE, RETIRE) with the semantics defined in Section 5.4.
- Token ID deduplication with configurable retention window as described in Section 11.3.

Level 3 — Extended Conformance

A Level 3 conformant implementation MUST implement all Level 1 and Level 2 requirements plus all of the following:

- Delta object computation, canonicalization, hashing, and storage for all UPDATE and MERGE operations, as defined in Section 5.6.
- Branching and merge operations with branch label uniqueness enforcement and version namespace isolation as defined in Section 5.5.
- Schema versioning and migration support, including major-version migration token issuance, as defined in Section 7.4.
- Schema deprecation support in the registry without schema removal, as defined in Section 7.5.
- Lineage integrity verification for complete historical chains including branch reconciliation, as defined in Section 5.7.
- All error codes in Appendix B with the specified HTTP equivalents returned in rejection responses.

10.2 Conformance Test Vectors

The following table provides a set of test vectors that implementations SHOULD use to validate their conformance. "PASS" indicates the implementation should accept the described token; "FAIL" indicates it must be rejected with the specified error condition.

#	Test Case Description	Expected Outcome	Failure Condition / Error Code	Applies To Level
TV-01	Valid genesis token with correct self-referential asset_id and correct digest	PASS — Accept token	N/A	1, 2, 3
TV-02	Token with digest computed using SHA-256 instead of SHA3-256	FAIL — Reject token	INVALID_DIGEST	1, 2, 3
TV-03	Token with payload keys not in Unicode code point order	FAIL — Reject token	INVALID_DIGEST (digest will not match)	1, 2, 3
TV-04	Token with float value serialized as `195.42000`	FAIL — Reject token	INVALID_DIGEST (digest will not match)	1, 2, 3
TV-05	Valid v2 UPDATE token with correct parent_digest referencing a verified v1 token	PASS — Accept token	N/A	1, 2, 3
TV-06	Token claiming version 3 with parent_digest of a version 1 token (gap of 1)	FAIL — Reject token	LINEAGE_GAP	2, 3
TV-07	Token with operation CREATE at version 3	FAIL — Reject token	INVALID_OPERATION	2, 3
TV-08	Token submitted after a RETIRE token for the same asset	FAIL — Reject token	LINEAGE_AFTER_RETIRE	2, 3
TV-09	Token with schema_id referencing a non-existent schema	FAIL — Reject token	SCHEMA_NOT_FOUND	2, 3
TV-10	Token with payload field violating schema type constraint	FAIL — Reject token	SCHEMA_VIOLATION	2, 3

#	Test Case Description	Expected Outcome	Failure Condition / Error Code	Applies To Level
TV-11	Token with duplicate token_id (seen within retention window)	FAIL — Reject token	DUPLICATE_TOKEN_ID	2, 3
TV-12	Token with urib_version not supported by the implementation	FAIL — Reject token	UNSUPPORTED_URIB_VERSION	1, 2, 3
TV-13	Valid UPDATE token with correct delta_ref digest	PASS — Accept token	N/A	3
TV-14	Token with BOM prefix in serialized byte stream	FAIL — Reject token	INVALID_DIGEST (digest will not match)	1, 2, 3

10.3 Non-Conformance Handling

Conformant implementations MUST adhere to the following non-conformance handling requirements:

- **Unconditional rejection:** Tokens with invalid digests MUST be rejected. There is no grace mode, lenient mode, or bypass mechanism for digest verification failure. Implementations MUST NOT provide configuration options that allow digest-invalid tokens to be accepted.
- **Unconditional rejection for schema violations:** Tokens failing schema validation MUST be rejected. Partial acceptance — accepting the token but discarding the invalid fields — is explicitly non-conformant.
- **Mandatory error reporting:** Every rejection MUST produce a structured rejection record containing at minimum: the `token_id` (if present and parseable), the UTC timestamp of the rejection, the error code (see Appendix B), the human-readable error description, and the specific field or location of the violation.
- **No silent acceptance:** Implementations MUST NOT silently discard tokens that violate this specification. All violations MUST be logged and the rejection reason MUST be available for audit retrieval.
- **Error propagation:** Rejection responses MUST be propagated to the submitting party with sufficient detail to diagnose and correct the violation. Opaque rejection messages ("invalid token") without detail are non-conformant.

11 Security Considerations

11.1 Digest Integrity

SHA3-256 provides 256-bit security against second-preimage attacks — meaning an adversary cannot find a different token that produces the same digest without performing approximately 2^{256} hash operations. This security level is considered computationally infeasible under current and near-future computational models, including quantum-enhanced attacks (SHA3-256 provides 128-bit post-quantum security under Grover's algorithm, which remains acceptable for the threat models applicable to URIB deployments).

Downgrading the canonical hash algorithm to any weaker algorithm (SHA-256, SHA-1, MD5, or any non-SHA3 construction) MUST NOT be done without a formal deprecation notice issued through the URIB governance process, accompanied by a migration plan that re-digests all existing tokens under the new algorithm and updates all lineage references accordingly. Any such downgrade constitutes a major version increment to this specification (i.e., URIB 2.0.0).

Implementations MUST NOT expose digest computation as an API endpoint that accepts arbitrary inputs without rate limiting, as an adversary could use such an endpoint to enumerate preimage candidates.

11.2 Schema Injection

The schema registry is a critical trust boundary. A malicious or compromised schema could declare fields with permissive constraints (e.g., unrestricted string fields where structured identifiers are expected), enabling injection of malformed data that passes schema validation. To mitigate schema injection risks:

- Schemas retrieved from a registry MUST be treated as trusted only if retrieved over a verified channel: TLS 1.2 or higher with certificate validation is the minimum acceptable transport. TLS with certificate pinning is RECOMMENDED for production deployments.
- Content-addressable schema storage — where the schema is stored and retrieved by its own SHA3-256 hash — is RECOMMENDED as it provides cryptographic proof that the retrieved schema has not been tampered with after publication.
- Unverified schemas — schemas retrieved over unverified channels, schemas whose content hash does not match a pre-registered expected value, or schemas from registries with expired TLS certificates — MUST be rejected. A token referencing an unverifiable schema MUST be rejected with `SCHEMA_NOT_FOUND`.
- Schema content MUST be treated as untrusted data: extension fields (e.g., `x-urib-mapping-table`) MUST be parsed with strict type checking. Malformed extension fields MUST cause schema load failure, not silent degradation.

11.3 Replay Attacks

A replay attack in the URIB context consists of re-submitting a previously accepted, valid token to a URIB endpoint, potentially triggering duplicate processing (e.g., double-crediting a financial instrument, re-issuing an identity credential, or re-processing a supply chain event).

The primary defense against replay attacks is the mandatory `token_id` deduplication requirement. Because every token carries a UUIDv7 `token_id` that is globally unique per token instance, any token submitted twice will carry the same `token_id` and MUST be rejected on the second submission with error code `DUPLICATE_TOKEN_ID`.

Implementations MUST maintain a deduplication store keyed on `token_id`. The retention window for the deduplication store SHOULD be configurable and SHOULD be set based on the maximum expected replay window for the deployment context. For high-security deployments, the retention window SHOULD be indefinite (permanent storage of all seen `token_id` values). For resource-constrained deployments, a minimum retention window of 7 days is RECOMMENDED.

UUIDv7's time-ordered structure enables efficient deduplication store pruning by timestamp, as UUIDs with timestamps outside the retention window can be systematically expired without affecting the security guarantee within the window.

11.4 Lineage Tampering

Lineage tampering is the attempt to alter an existing token's payload after its digest has been computed and accepted, or to insert spurious tokens into a lineage chain to fabricate a false asset history. The URIB integrity model provides strong guarantees against both attack vectors:

- **Post-digest mutation:** Any mutation to any field of a token after digest computation will cause the recomputed digest to differ from the stored `digest` field, and the token will be rejected on next verification. Because the digest is stored with the token and verified on every receipt, there is no silent window during which a mutated token can be accepted.
- **Chain insertion:** Inserting a spurious token into the middle of a lineage chain requires recomputing all digests from the insertion point forward (since each token's digest depends on its parent's digest). This requires breaking the SHA3-256 second-preimage resistance for every affected token — computationally infeasible at 256-bit security.
- **Chain truncation:** Attempting to remove a token from the chain will create a parent digest mismatch in the token that followed the removed token, causing lineage verification to fail.

Implementations MUST verify digests before trusting lineage claims. Lineage verification (Section 5.7) MUST NOT be treated as optional by any Level 2 or Level 3 conformant implementation.

11.5 Retire Bypass

The RETIRE operation's finality guarantee is a security property, not merely a business rule. A malicious or compromised client may attempt to submit token versions beyond a RETIRE token in order to resurrect an end-of-life asset — for example, to reactivate a retired financial instrument or re-establish a revoked identity credential.

RETIRE finality MUST be enforced server-side at the ingest layer. Client-submitted tokens claiming version numbers after a verified RETIRE token for the same `asset_id` MUST be rejected at ingest with error code `LINEAGE_AFTER_RETIRE`, regardless of the operation type of the submitted token. The submitted token MUST NOT be passed to digest verification, schema validation, or any downstream processor.

Implementations MUST maintain an indexed record of all `asset_id` values for which a RETIRE token has been accepted. This index MUST be consulted before accepting any token for a given `asset_id`. The RETIRE index MUST be protected against unauthorized modification — write access to the RETIRE index MUST be restricted to the ingest system and MUST be audited.

12 Appendices

Appendix A — Reserved Domain Identifiers

The following domain identifiers are reserved by the URIB platform and MUST NOT be used for custom domains outside their defined contexts. Custom domain namespaces MUST be registered with the URIB schema registry governance process before use in production tokens.

Domain Identifier	Domain Description	Typical Schema Names
`finance.instrument`	Financial instruments (equities, bonds, derivatives, funds)	equity_token, bond_token, derivative_contract, fund_unit
`finance.transaction`	Financial transactions (trades, transfers, settlements, payments)	trade_record, transfer_event, settlement_instruction, payment_order
`health.record`	Healthcare records (patient encounters, clinical observations, prescriptions)	patient_encounter, clinical_observation, medication_order, lab_result
`health.claim`	Healthcare insurance claims and adjudications	insurance_claim, adjudication_result, prior_authorization
`identity.credential`	Digital identity credentials and verifiable presentations	verifiable_id, passport_credential, driver_license, employee_badge
`identity.session`	Authentication and authorization session records	auth_session, access_token_record, consent_grant
`supply.asset`	Supply chain physical and digital assets (inventory, equipment, materials)	inventory_item, equipment_record, raw_material, finished_good
`supply.shipment`	Supply chain shipment and logistics records	shipment_manifest, customs_declaration, delivery_event, bill_of_lading
`iot.telemetry`	IoT device telemetry readings and sensor data	device_reading, sensor_snapshot, telemetry_batch, alert_event

Domain Identifier	Domain Description	Typical Schema Names
`iot.device`	IoT device registry and configuration records	device_registration, firmware_version, configuration_state, device_twin
`legal.contract`	Legal contracts and agreements in structured form	service_agreement, nda_record, licensing_contract, terms_acceptance
`legal.signature`	Digital signature records and audit-trail events	signature_event, notarization_record, witness_attestation

Appendix B — Error Codes

The following error codes MUST be used in rejection responses by all Level 3 conformant implementations, and SHOULD be used by Level 1 and Level 2 conformant implementations. The HTTP equivalent column provides the recommended HTTP status code when the rejection is communicated over an HTTP API.

Code	Name	Description	HTTP Equivalent	Applies At Step
`E001`	INVALID_DIGEST	The token's `digest` field does not match the SHA3-256 hash of the token's canonical serialization with the digest placeholder substituted. Indicates tampering, serialization error, or wrong hash algorithm.	422 Unprocessable Entity	Digest verification (§4.3)
`E002`	SCHEMA_NOT_FOUND	The schema referenced by `meta.schema_id` could not be resolved in the schema registry. Either the schema has not been published or the schema_id is malformed.	422 Unprocessable Entity	Schema resolution (§7.6)
`E003`	SCHEMA_VIOLATION	The token's `payload` fails validation against the schema referenced by `meta.schema_id`. The rejection response MUST include the full list of validation errors from the JSON Schema validator.	422 Unprocessable Entity	Payload validation (§7.6)
`E004`	LINEAGE_GAP	The token's `lineage.version` is not exactly one greater than the version of the token identified by `lineage.parent_digest`. A version gap has been detected in the lineage chain.	409 Conflict	Lineage verification (§5.3, §5.7)
`E005`	LINEAGE_AFTER_RETIRE	A token has been submitted for an asset	409 Conflict	Ingest pre-check

Code	Name	Description	HTTP Equivalent	Applies At Step
		whose mainline lineage chain has already been terminated by a RETIRE operation. No further tokens may be accepted for this asset on the mainline.		(§5.4, §11.5)
E006	INVALID_OPERATION	The value of `lineage.operation` is semantically invalid in context — for example, operation CREATE at version > 1, or an unrecognized operation string.	422 Unprocessable Entity	CDM validation (§2.5, §5.4)
E007	DUPLICATE_TOKEN_ID	The `meta.token_id` value has been seen previously within the deduplication retention window. This token is a duplicate submission or a replay attempt.	409 Conflict	Ingest pre-check (§11.3)
E008	UNSUPPORTED_URIB_VERSION	The `meta.urib_version` value specifies a version of this specification that the receiving implementation does not support. The token cannot be processed without version compatibility guarantees.	422 Unprocessable Entity	CDM validation (§2.3)
E009	CDM_STRUCTURE_VIOLATION	The token does not conform to the Canonical Document Model structure — for example, a missing top-level block, an unexpected additional top-level key, or a missing mandatory field within a block.	400 Bad Request	CDM validation (§2.1, §2.2)
E010	ASSET_ID_MISMATCH	The `meta.asset_id` value does not match the digest of the genesis token in the resolved lineage chain, or at genesis does not equal the token's own digest.	422 Unprocessable Entity	Lineage verification (§4.4, §5.2)

Code	Name	Description	HTTP Equivalent	Applies At Step
`E011`	SCHEMA_DEPRECATED	The schema referenced by `meta.schema_id` has been marked deprecated in the schema registry. Used when implementation policy requires rejection of deprecated-schema tokens (otherwise a warning is issued and processing continues).	422 Unprocessable Entity	Schema resolution (§7.5, §7.6)

Appendix C — Changelog

Version	Date	Status	Changes
1.0.0	July 2026	NORMATIVE	Initial release. All sections normative. Defines: the URIB Canonical Document Model (CDM); the nine-step deterministic serialization algorithm; SHA3-256 canonical hashing with digest computation and verification procedures; the four lineage operations (CREATE, UPDATE, MERGE, RETIRE); reversible mapping guarantees and primitive type mapping table; JSON Schema draft-07 schema-binding requirements with URIB extension vocabulary (`x-urib-*`); three conformance levels (Basic, Full, Extended); eleven error codes; twelve reserved domain identifiers. No predecessor versions.